



Guide Docker pour StatDirm

Ce guide explique comment utiliser Docker pour développer et déployer l'application StatDirm.



Prérequis

- Docker Desktop installé (ou Docker Engine + Docker Compose)
- Au moins 2 Go d'espace disque disponible



Démarrage rapide

Développement

Pour lancer l'application en mode développement avec hot-reload :

```
# Option 1: Utiliser docker-compose.dev.yml
docker-compose -f docker-compose.dev.yml up

# Option 2: Utiliser le fichier principal avec le service dev
docker-compose up app-dev

# Option 3: Build et run manuel
docker build --target development -t statdirm-dev .
docker run -p 3000:3000 -v $(pwd):/app statdirm-dev
```

L'application sera accessible sur <http://localhost:3000>

Production

Pour construire et lancer l'application en production :

```
# Option 1: Utiliser docker-compose.prod.yml
docker-compose -f docker-compose.prod.yml up -d

# Option 2: Utiliser le fichier principal avec le service prod
docker-compose up -d app-prod

# Option 3: Build et run manuel
docker build --target production -t statdirm-prod .
docker run -d -p 8080:80 --name statdirm-prod statdirm-prod
```

L'application sera accessible sur <http://localhost:8080>

Commandes utiles

Développement

```
# Démarrer en arrière-plan
docker-compose -f docker-compose.dev.yml up -d

# Voir les logs
docker-compose -f docker-compose.dev.yml logs -f

# Arrêter
docker-compose -f docker-compose.dev.yml down

# Rebuild après modification du Dockerfile
docker-compose -f docker-compose.dev.yml up --build
```

Production

```
# Démarrer en arrière-plan
docker-compose -f docker-compose.prod.yml up -d

# Voir les logs
docker-compose -f docker-compose.prod.yml logs -f app

# Arrêter
docker-compose -f docker-compose.prod.yml down

# Rebuild
docker-compose -f docker-compose.prod.yml up --build -d
```

Scripts Python (conversion Excel)

```
# Convertir un fichier Excel en JSON
docker-compose run --rm converter python3 scripts/convert_excel_to_json.py
trdata/votre_fichier.xlsx

# Analyser un fichier Excel
docker-compose run --rm converter python3 scripts/analyze_excel.py
```

Structure Docker

Dockerfile multi-stage

Le Dockerfile utilise une approche multi-stage pour optimiser la taille de l'image :

1. **Stage** `builder` : Construit l'application React avec Node.js
2. **Stage** `production` : Serve l'application avec Nginx (image légère)
3. **Stage** `development` : Environnement de développement avec hot-reload

Services Docker Compose

- **app-dev** : Application en mode développement
- **app-prod** : Application en mode production avec Nginx
- **converter** : Service Python pour convertir les fichiers Excel (profile: tools)

Mise à jour des données

Pour mettre à jour les données JSON depuis Excel :

```
# Méthode 1: Utiliser le service converter
docker-compose run --rm converter python3 scripts/convert_excel_to_json.py
trdata/Interface_Effectifs_DIRM_Central_V6_corrected.xlsx

# Méthode 2: Exécuter directement dans le conteneur
docker exec -it statdirm-dev python3 scripts/convert_excel_to_json.py trdata/
votre_fichier.xlsx
```

Les données seront automatiquement disponibles dans l'application après rechargement.

Développement

Hot-reload

En mode développement, les modifications du code sont automatiquement reflétées grâce au montage de volume :

```
docker-compose -f docker-compose.dev.yml up
```

Modifiez vos fichiers localement, les changements seront visibles immédiatement dans le navigateur.

Installation de nouvelles dépendances

```
# Installer une nouvelle dépendance
docker exec -it statdirm-dev npm install nom-du-package

# Ou depuis votre machine locale (le volume est monté)
npm install nom-du-package
```

Déploiement

Build pour production

```
# Build l'image
docker build --target production -t statdirm:latest .

# Tag pour un registry (optionnel)
docker tag statdirm:latest votre-registry/statdirm:v1.0.0

# Push vers le registry (optionnel)
docker push votre-registry/statdirm:v1.0.0
```

Variables d'environnement

Créez un fichier `.env` pour les variables d'environnement :

```
NODE_ENV=production
VITE_API_URL=https://api.example.com
```

Puis utilisez-le :

```
docker-compose -f docker-compose.prod.yml --env-file .env up -d
```

Dépannage

Le conteneur ne démarre pas

```
# Vérifier les logs
docker-compose logs app

# Vérifier l'état
docker-compose ps
```

```
# Rebuild complet
docker-compose down -v
docker-compose up --build
```

Port déjà utilisé

Si le port 3000 ou 8080 est déjà utilisé, modifiez le mapping dans `docker-compose.yml` :

```
ports:
  - "3001:3000" # Utiliser le port 3001 au lieu de 3000
```

Problèmes de permissions

Sur Linux, vous pourriez avoir besoin de :

```
sudo docker-compose up
```

Ou ajouter votre utilisateur au groupe docker :

```
sudo usermod -aG docker $USER
# Puis reconnectez-vous
```

Nettoyer les images et conteneurs

```
# Arrêter et supprimer les conteneurs
docker-compose down

# Supprimer aussi les volumes
docker-compose down -v

# Nettoyer les images non utilisées
docker image prune -a

# Nettoyer complètement (attention !)
docker system prune -a --volumes
```

Monitoring

Vérifier l'utilisation des ressources

```
# Stats en temps réel
docker stats
```

```
# Pour un conteneur spécifique
docker stats statdirm-prod
```

Healthcheck

Le service de production inclut un healthcheck automatique. Vérifier le statut :

```
docker inspect --format='{{.State.Health.Status}}' statdirm-prod
```

Sécurité

Bonnes pratiques

1. **Ne pas exposer les données sensibles** : Les fichiers Excel ne sont pas inclus dans l'image Docker par défaut
2. **Utiliser des secrets** : Pour les variables sensibles, utilisez Docker secrets ou un gestionnaire de secrets
3. **Mettre à jour régulièrement** : Gardez les images de base à jour (`node:20-alpine` , `nginx:alpine`)
4. **Scan de vulnérabilités** : Utilisez `docker scan` pour vérifier les vulnérabilités

```
docker scan statdirm:latest
```

Notes

- Les données JSON doivent être présentes dans `src/data/agents.json` avant le build de production
- En développement, les modifications du code sont reflétées en temps réel grâce aux volumes
- Le service `converter` utilise le profile `tools` et n'est pas démarré par défaut

Support

En cas de problème, vérifiez :

1. Les logs : `docker-compose logs`
2. L'état des conteneurs : `docker-compose ps`

3. La configuration : `docker-compose config`